Copy-Move Forgery Detection using a Siamese Swap-Test Quantum Neural Network

Dr. Baskar A
(Assistant Professor)
Department of Computer Science
Amrita University
Coimbatore, India

Naman Yadav
Department of Computer Science
Amrita University
Coimbatore, India

Adiseshan Ramanan
Department of Computer Science
Amrita University
Coimbatore, India

***Abstract*—Copy-move forgery is a common form of digital image manipulation in which a region of an image is copied and pasted into another location within the same image to conceal, duplicate, or alter visual information. Because the source and pasted regions share identical sensor noise, illumination, and compression history, such tampering is difficult to expose through visual inspection or purely statistical forensic cues, which makes reliable detection important for digital forensics, journalism, legal evidence, and the verification of image integrity on social media. Existing copy-move forgery detection (CMFD) methods commonly rely on handcrafted block descriptors or keypoint matching, both of which degrade under geometric transformation and post-processing, or on deep convolutional architectures, which require large labelled datasets and remain prone to false matches between visually similar but genuinely distinct regions. Natural self-similarity in sky, foliage, and brickwork is a persistent source of this kind of error, one that pairwise similarity scoring alone cannot resolve without an explicit model of spatial consistency between matched regions. A further gap is that quantum machine learning remains almost entirely unexplored for forensic pairwise-matching tasks such as CMFD, and no prior work, to our knowledge, combines a Siamese comparison architecture with a quantum swap-test similarity readout and an in-circuit encoding of spatial displacement. To address this gap, this work proposes a Siamese Swap-Test Quantum Neural Network (ST-QNN) for copy-move forgery detection. Candidate image patches are tiled from the input image, and for every admissible pair, they are amplitude-encoded into two quantum registers driven by a single shared, tied-weight set of trainable rotation angles, so that both patches are processed through parameter-identical pathways and yield directly comparable quantum states. A data re-uploading scheme interleaves the raw pixel values into the rotation angles at every variational layer; this is required to keep the comparison trainable rather than mathematically degenerate under a swap test. A swap test then estimates the overlap between the two patches' quantum states via a single ancilla-qubit measurement, functioning as a quantum, trainable analogue of a learned similarity metric. To reduce the false matches caused by natural self-similarity, the spatial offset between each patch pair is angle-encoded into a small offset register and folded into the same ancilla readout through trainable controlled rotations, so that the measured similarity is conditioned on geometric displacement rather than pixel content alone. This gives an in-circuit, quantum-native counterpart to the classical spatial-consistency voting used in traditional block-matching pipelines. The proposed approach is evaluated primarily on the CoMoFoD benchmark of authentic and copy-move forged images with pixel-level ground-truth masks, with the COVERAGE dataset of visually similar genuine objects used as an adversarial stress test, and with self-supervised synthetic copy-move pairs used for training, since neither benchmark provides patch-level correspondence labels. The proposed framework is assessed using standard detection and localization metrics, namely precision, recall, F1-score, and intersection-over-union, and its behaviour is examined against classical similarity baselines and representative existing CMFD methods. This evaluation is intended to characterize how effectively a fully quantum decision path, combining amplitude-encoded raw pixels, a trainable swap-test similarity readout, and an in-circuit spatial-offset register, can identify duplicated regions and exploit spatial consistency for copy-move forgery detection, and to clarify the practical trade-offs such an architecture presents relative to classical and deep-learning approaches.**

***Index Terms*—Copy-move forgery, image forgery detection, digital image forensics, Siamese neural network, quantum neural network, quantum machine learning, swap test, spatial offset consistency, quantum-classical learning.**

I. Introduction

A. Background

Digital images have become an essential medium for communication, information sharing, journalism, scientific documentation, and digital content creation. The widespread availability of image-editing software and, increasingly, generative image-editing tools has made it difficult to determine whether an image accurately represents the original scene it depicts. **Digital image forgery** broadly refers to any deliberate manipulation of an image's visual content that misrepresents the scene it purports to show, and is commonly grouped into three categories: *splicing*, in which content from one image is inserted into another; *retouching*, in which existing content is edited, smoothed, or removed without introducing external material; and *copy-move forgery*, in which a region of an image is copied and pasted onto another location within the *same* image. Digital image forensics has therefore become an important research area, tasked with identifying unauthorized modifications and establishing the authenticity and integrity of digital visual content, particularly as manipulated images circulate rapidly through social media and news channels where their apparent authenticity carries real consequences.

Among these categories, **copy-move forgery** is one of the most common and challenging forms of image manipulation to detect. A copy-move operation duplicates a region of an image, whether to conceal an object, fabricate additional instances of it, or reinforce and exaggerate visual content, and pastes

it elsewhere in the same frame. This is frequently followed by post-processing such as blending, rotation, scaling, or compression to disguise the seam. Because the source and duplicated regions originate from the same image, they inherently share the same sensor noise pattern, illumination conditions, color response, and compression history. That is precisely what makes copy-move forgery both an effective manipulation technique and a difficult one to expose: the statistical and metadata-based cues that splicing detectors rely on, such as inconsistent noise or compression artifacts between regions taken from different source images, are simply absent, since there is only ever one source image involved.

B. Problem Statement

The primary objective of copy-move forgery detection is to identify duplicated image regions, determine whether their spatial relationship is consistent with a copy-and-paste operation, and localize the affected regions. This is difficult for two compounding reasons. First, because copy-move forgery leaves no cross-image statistical inconsistency to exploit, detection must instead be framed as an *intra-image* similarity problem: searching the image for regions that resemble each other more closely than chance would predict. Second, and more fundamentally, natural images are *self-similar*. Skies, foliage, brickwork, water, and other textured or smooth regions produce large numbers of patches that are genuinely, coincidentally similar to one another without ever having been copy-pasted. A detector built purely on pairwise region similarity therefore cannot, by similarity alone, distinguish a genuine duplication event from this background of natural self-similarity. Some additional structural signal is required to separate the two, most commonly the observation that copy-moved regions share a single, consistent spatial displacement while coincidental look-alikes scatter across many displacements.

This difficulty is further compounded when the pasted region has been deliberately or incidentally altered before or after pasting, since a robust detector must remain effective under the following:

- **Rotation**: the pasted region rotated relative to its source, which permutes low-level pixel-based descriptors and defeats detectors that assume translation-only correspondence;
- **Scaling**: the pasted region resized relative to its source, altering the spatial frequency content local descriptors rely on;
- **Compression**: lossy re-encoding (e.g., JPEG) of the forged image, which introduces block artifacts that can mask or mimic genuine copy-move evidence;
- **Noise**: sensor or synthetic noise added after pasting, which perturbs pixel-level similarity between the source and duplicated regions;
- **Blur**: smoothing applied to disguise the paste boundary, which suppresses the high-frequency detail many descriptors depend on.

An effective CMFD method must therefore combine a similarity mechanism that is robust to these degradations with a mechanism for distinguishing genuine duplication from coincidental resemblance.

C. Existing Methods

Traditional CMFD approaches fall broadly into two families. *Block-based* methods divide the image into overlapping blocks, compute a descriptor for each block (commonly discrete cosine transform, or DCT, coefficients), and identify duplicated regions through lexicographic sorting and matching of similar descriptors, as in the foundational method of Fridrich et al. [1]. *Keypoint-based* methods instead detect sparse, distinctive local features, most commonly SIFT keypoints, and match their descriptors across the image, as in the SIFT-based forensic method of Amerini et al. [2], which additionally clusters matched keypoints by their relative geometric transformation to recover the copy-move operation and suppress spurious matches. More recently, deep learning methods have been investigated to learn discriminative region representations directly from data rather than relying on handcrafted descriptors. BusterNet [5], for example, uses a two-branch convolutional architecture that jointly localizes manipulated regions and assesses visual similarity between candidate regions, fusing the two signals to both detect and localize source and target copy-move regions in an end-to-end trainable manner.

Each family carries characteristic limitations. Block-based descriptors are sensitive to rotation and scaling of the pasted region and incur substantial computational cost when matching is performed exhaustively across all blocks. Keypoint-based methods are more robust to geometric transformation but produce very few or no keypoints in smooth, low-texture regions, which are precisely the homogeneous areas (sky, walls, water) most prone to being copy-moved to conceal something, so detection coverage suffers there. Deep learning methods require substantial labelled training data, incur non-trivial computational cost, and, most importantly for this work, remain vulnerable to false matches between regions that are visually similar but not the product of a copy-move operation, since a learned similarity signal alone does not guarantee that matched regions arose from duplication rather than coincidence. Across all three families, **patch or region similarity estimation is the common core primitive** that every CMFD pipeline is ultimately built on top of. Detection is fundamentally the question of whether a given region matches another region elsewhere in the image, which is why the quality, robustness, and trainability of that similarity estimate, and how it is combined with spatial-consistency evidence, forms the central design question this work addresses.

D. Research Gap

Two gaps motivate this work. First, within classical CMFD, similarity estimation and spatial-consistency filtering are almost always implemented as two *separate* stages: a similarity or matching stage followed by a hand-designed voting or clustering stage (e.g., a Hough-style vote over candidate displacement vectors) that is not learned jointly with the

similarity metric itself. Second, and more specifically to this work, **quantum machine learning (QML) remains largely unexplored for forensic pairwise-matching tasks such as CMFD**. The great majority of QML research to date targets generic classification benchmarks [7]. Quantum primitives well suited to similarity estimation do exist, most notably the swap test [8], which estimates the overlap between two quantum states via a single ancilla measurement, and quantum feature-space or kernel methods that interpret data encoding as a nonlinear feature map into Hilbert space [10], but to our knowledge no prior work has combined a Siamese (shared-weight) comparison architecture with a swap-test similarity readout, still less applied such an architecture to copy-move forgery detection or attempted to fold spatial-consistency evidence into the quantum circuit itself rather than handling it with a separate classical stage. A new method is therefore required both to test whether a trainable quantum similarity estimator is viable for this task at all, and to establish whether the spatial-consistency signal that classical CMFD depends on can be represented and exploited from inside the quantum circuit.

E. Proposed Idea

This work proposes a **Siamese Swap-Test Quantum Neural Network (ST-QNN)** for copy-move forgery detection. The input image is tiled into fixed-size patches, and every admissible pair of patches (subject to a minimum spatial separation) is scored by a single shared quantum circuit. Each patch's raw pixel values are amplitude-encoded into their own quantum register, so that a patch of 2^n pixels is represented using only n qubits. Both registers are then processed by *identical*, tied-weight variational layers, the quantum analogue of a Siamese network's shared-weight branches, in which the pixel data is re-uploaded into the rotation angles at every layer following the data re-uploading approach of Pérez-Salinas et al. [9]. This is a structural requirement rather than a stylistic choice: a fixed amplitude embedding followed by a data-*independent* tied unitary is provably degenerate under a swap test, since the unitary cancels against its own adjoint on both branches, leaving zero gradient with respect to the trainable weights. A swap test between the two registers then estimates their state overlap via a single ancilla measurement, giving a trainable, quantum-native similarity score in place of a classical distance metric. To address the self-similarity problem identified above, the spatial offset between the two patches is angle-encoded into an additional small register and coupled into the same ancilla readout through trainable controlled rotations, so the circuit's decision is conditioned on *both* content similarity and spatial displacement within a single measurement, rather than through a separate classical voting stage applied after the fact.

The main novelty of this work is twofold: (i) to our knowledge, this is the first application of a Siamese swap-test quantum architecture to copy-move forgery detection, and (ii) the spatial-consistency signal that classical CMFD pipelines compute externally is instead encoded and processed *inside* the quantum circuit, via the offset register and its trainable coupling into the swap-test readout. The main contributions of this work are summarized as follows:

- A Siamese Quantum Neural Network architecture is proposed for copy-move forgery detection, in which two patches are compared through parameter-tied, data-re-uploading variational quantum circuits.
- A swap-test-based quantum similarity mechanism is used to compare the amplitude-encoded quantum states of candidate image patches, and the necessity of data re-uploading for this mechanism to be trainable at all is established both analytically and empirically (via measured gradient magnitude).
- An in-circuit spatial offset register with trainable controlled-rotation coupling is introduced as a quantum-native counterpart to the classical spatial-consistency vote used in traditional block-matching CMFD pipelines.
- The proposed quantum architecture is experimentally evaluated on the CoMoFoD benchmark [3], with the COVERAGE dataset [4] used as an adversarial stress test against visually similar but genuine (non-duplicated) objects, and with self-supervised synthetic copy-move pairs used for training.
- The performance of the proposed method is compared against classical similarity baselines and, where applicable, existing block-based, keypoint-based, and deep-learning CMFD methods [1], [2], [5].

The remainder of this paper is organized as follows. Section II presents the proposed Siamese Swap-Test Quantum Neural Network architecture in detail. Section III describes the experimental setup and datasets. Section IV presents the experimental results and comparison with existing approaches. Finally, Section V concludes the paper and discusses potential directions for future research.

II. Proposed Work

A. Overall System

The system accepts a single digital photograph as input, whether a JPEG or PNG file, in colour or already in grayscale, at whatever native resolution the source file happens to have, and returns two things: an image-level verdict of *forged* or *authentic*, and, when the verdict is forged, a localization of the suspected duplicated region expressed both as a pixel-level mask and as a bounding box drawn on the original image. Nothing about the input format is assumed beyond it being decodable as a raster image; colour images are converted internally (Section II-B), and images of any aspect ratio or size are brought to a common working resolution before a single patch is cut.

Between that input and that output, the pipeline passes through nine named stages, each covered in its own subsection below, and it is important to be precise about which of them run as ordinary classical code and which run *inside* a single quantum circuit invocation, since the two are easy to conflate in a summary diagram. Three stages are purely classical and run once per image, before any quantum computation begins:

the image is standardized (**Image Pre-processing**); cut into a grid of small, non-overlapping squares (**Patch Extraction**); and each square is reduced, once, to a plain normalized numerical vector representing its own pixel content (**Feature Extraction**). Four stages then happen together, inside one shared quantum circuit that is invoked separately for every candidate pair of patches: the two patches' vectors are loaded onto qubits (**Quantum Feature Encoding**); both registers are processed by parameter-tied variational layers (**Siamese QNN**); the two resulting quantum states are compared (**Swap Test**); and the spatial displacement between the two patches is folded into that same comparison rather than applied afterward (**Spatial Offset Consistency**), so that a single circuit execution returns one number that already reflects both content similarity and spatial plausibility, not four separate hand-offs. Two final classical stages then operate on the whole collection of these circuit outputs at once, after every pair in the image has been scored: pairs above the similarity threshold are kept as **Patch Matches**, and those matches are aggregated into a mask, a heatmap, and an image-level verdict (**Forgery Detection**). This ordering matters specifically because it is not the generic "compare, then vote" pipeline used by classical block-matching detectors: there, spatial consistency is checked *after* matching, as an external filter over an already-fixed set of matches, whereas here the spatial term is wired into the very same measurement that produces the similarity score in the first place, and nothing resembling a separate classical voting stage runs at any point. The diagram below groups the nine stages this way, with the middle row denoting the one circuit that is actually executed, once per pair:

Input Image $\rightarrow$ **Image Pre-processing** $\rightarrow$ **Patch Extraction** $\rightarrow$ **Feature Extraction**
$\downarrow$ *(per admissible pair, one circuit call)*
Quantum Feature Encoding $\rightarrow$ **Siamese QNN** $\rightarrow$ **Swap Test** $\leftrightarrow$ **Spatial Offset Consistency**
$\downarrow$ *(after all pairs are scored)*
Patch Matching $\rightarrow$ **Forgery Detection**

Detection works by exhaustive comparison rather than by first searching for likely candidates: every admissible pair of patches drawn from the tiled grid, 32,640 pairs per image at the default configuration used throughout this paper and derived in Section II-C, is scored independently by the same circuit, and only once every pair has been scored does the system look at the resulting set of matches as a whole. A single patch can appear in many pairs at once, since a 16×16 grid compares every patch against up to 255 others, so a patch sitting inside a genuinely duplicated region is expected to accumulate matches against every patch in the corresponding source region, whereas a patch that merely resembles something by chance is expected to pick up, at most, an isolated match or two. The forged-region mask built in Section II-I follows directly from this accumulation: any patch taking part in at least one surviving match is marked, so the more completely a region has been duplicated, the more fully it is expected to appear in the output. Everything upstream of that final aggregation, including resizing the image, laying out the patch grid, and, purely so a human viewer sees a rectangle rather than a scattered mask, grouping marked pixels into a bounding box, is ordinary classical bookkeeping that decides nothing about which patches match. That decision belongs entirely to the circuit output described in the sections that follow.

It helps to make this concrete before the per-stage detail below. In one full end-to-end run of this exact pipeline (Section IV-A), 32,640 pairs from a single 128×128 image were scored one by one through the circuit described in Sections II-E–II-H, of which 1,509 cleared the similarity threshold and were carried forward into the aggregation stage of Section II-I. Every one of those 1,509 numbers, and the 31,131 that fell below threshold and were discarded, is the direct output of one circuit invocation with no classical adjustment in between. The pipeline diagram above is therefore not a simplification of a more complicated real process; it is a literal description of how that specific 32,640-pair run, and every other result in this paper, was actually produced.

B. Image Pre-processing

Figure 1 previews the whole classical journey from photograph to quantum-ready vectors that Sections II-B–II-E walk through step by step: grayscale conversion and resizing, tiling into patches, flattening and L2-normalizing each patch, and forming admissible pairs.

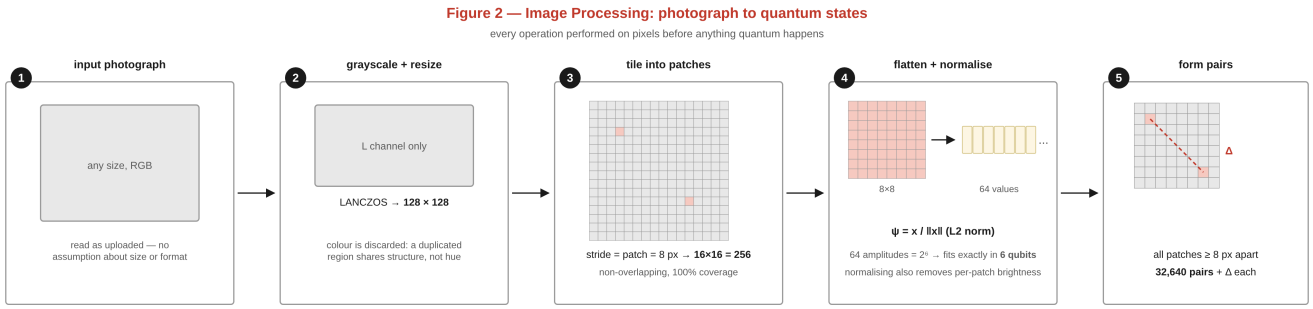


Fig. 1. Image pre-processing pipeline: a photograph of any size and colour depth is converted to grayscale, resized to 128×128 via Lanczos resampling, tiled into 256 non-overlapping 8×8 patches, flattened and L2-normalized into 64-value amplitude vectors, and formed into admissible pairs subject to the minimum-separation rule of Section II-C.

Before any patch is extracted, the input image is converted to grayscale and resized to a fixed square working resolution, taken here as 128×128 pixels, using Lanczos resampling rather than a cheaper kernel such as nearest-neighbour or bilinear interpolation. Lanczos resampling was chosen specifically because it preserves fine edges and texture better under both upscaling and downscaling than those simpler kernels; since a copy-move forgery can occupy a small fraction of the frame and the circuit has no source of information about a patch beyond the pixel values it is given, destroying fine detail at the resizing step would remove evidence no later stage could recover. The practical difference between these kernels is a matter of how many neighbouring source pixels each output pixel is computed from: nearest-neighbour resampling uses exactly one, bilinear interpolation uses a 2×2 neighbourhood, and Lanczos resampling convolves with a windowed sinc function over a wider neighbourhood (commonly a 3- or 4-lobe window), which approximates an ideal low-pass filter far more closely than a linear ramp does and is correspondingly better at suppressing the aliasing that shows up as spurious high-frequency texture once an image is downscaled. Since the swap test of Section II-G compares patches on their raw pixel content with no descriptor standing between the image and the circuit, any aliasing artefact introduced at this stage would be indistinguishable, from the circuit's point of view, from genuine texture. A resampling-kernel choice that a classical descriptor-based pipeline might not even need to think about therefore has a direct, first-order effect on what the circuit is comparing here. The image is resized directly to a square without preserving its original aspect ratio, which is a deliberate simplification rather than an oversight: the detector reasons about small local patches rather than whole-object shape, so the resulting mild geometric distortion has no bearing on whether two patches' pixel content matches.

Grayscale conversion is used rather than processing the three colour channels separately, applying the standard luma-weighted average $L = 0.299R + 0.587G + 0.114B$. This choice is dictated directly by the qubit budget: amplitude encoding (Section II-E) maps every value in a patch onto one amplitude of a quantum state, so a colour patch would need either three times as many amplitudes packed into one register or a separate register per channel, both of which push the simulated qubit count for a single pair comparison well past what is practical once tens of thousands of pairs must be scored per image. Working in grayscale keeps that count to the six qubits per register used throughout this work (Table IV).

The specific coefficients above follow the ITU-R BT.601 luma standard rather than the newer BT.709 weighting used for HDTV; the two differ by only a few percent per channel and neither has a documented advantage for this task, but BT.601 is what the underlying image library's default grayscale conversion applies, and keeping that default avoids introducing an extra, unmotivated design choice into the pipeline. When the source image is already single-channel, a scan or a photograph already saved in grayscale, this step is a no-op rather than a forced conversion: there is no colour information to discard, so the raw pixel values pass through unchanged. To make the scale of the overall reduction concrete, a typical modern smartphone photograph, at roughly 4000×3000 pixels, is reduced by grayscale conversion and resizing to 128×128 before a single patch is ever compared: a linear reduction of roughly 30-fold along each axis and an area reduction of more than 700-fold. Everything the swap test can possibly detect is drawn from what survives that reduction, which is precisely why Lanczos resampling, rather than a cheaper kernel, was chosen for it.

The fixed working resolution is chosen so that the patch grid tiles the image exactly, with no leftover strip along either edge, and so that the number of patches, and with it the number of pairs the circuit has to score, stays computationally affordable. Because pair count grows quadratically with patch count, going from a 128×128 working resolution to 256×256 quadruples the patch count under the same 8×8 tiling and, since every patch is compared against every other, multiplies the number of pairs, and with it the run time, by roughly a factor of sixteen: from 32,640 pairs to well over half a million. Pixel intensities are then rescaled from the usual $[0, 255]$ integer range to floating-point values in $[0, 1]$. This step is a standard classical normalization and should not be confused with the L2 normalization applied later to each

individual patch (Section II-D), which exists for an entirely different reason: a quantum state vector must have unit norm, whereas the $[0, 1]$ rescaling here is only meant to put pixel intensities on a numerically convenient scale before the image is cut into patches. Together, these three operations solve the same underlying problem. Every later stage of this pipeline assumes a fixed-size, fixed-range, single-channel input, and a real photograph, as supplied by a user, arrives in none of those forms until pre-processing has run.

C. Patch Extraction

The image is divided into patches because copy-move forgery is, fundamentally, a local phenomenon: a duplicated region occupies only a fraction of the frame, so comparing an entire image against itself as a single unit would neither be computationally sensible nor say anything about *where* a duplication occurred. Patches are the smallest unit the system reasons about, and the resolution at which the final mask can localize a forged region is exactly the patch size.

Each patch is 8×8 pixels. This value is not chosen for image-processing reasons alone; it is chosen jointly with the quantum encoding in Section II-E, since a $P \times P$ patch requires $\lceil \log_2(P^2) \rceil$ qubits under amplitude encoding, and that qubit count directly sets the simulated cost of every one of the tens of thousands of pair comparisons an image requires. An 8×8 patch needs 6 qubits per register; the next standard size up, 16×16, needs 8, and because the classical simulation cost of these circuits grows exponentially with qubit count, moving to 16×16 patches increases the per-pair cost by roughly a factor of 23. Eight pixels is therefore the largest patch size that keeps exhaustive, per-pair evaluation affordable at the working resolution used in this paper, while remaining small enough that a forged region spanning only a few patches is not missed entirely.

Patches are extracted with a stride equal to the patch size, so that the resulting grid tiles the image edge to edge with no gaps and no overlap; overlapping extraction (a stride smaller than the patch size) is deliberately avoided. The choice is not merely a matter of convenience: a stride *larger* than the patch size, which leaves gaps in the grid, was found to break detection outright rather than merely reduce its resolution. At a 256×256 working resolution with a stride of 32 against an 8×8 patch, only 3 of the 64 sampled patches fell inside a forged region covering roughly 2% of the image area, and the small number of samples simply missed the manipulated pixels entirely. Full, non-overlapping tiling is the smallest stride that guarantees every pixel in the image is covered by exactly one patch, which is the coverage guarantee the rest of the pipeline depends on. At 128×128 working resolution this yields a 16×16 grid, or 256 patches per image.

Every one of these 256 patches is, in principle, a candidate for comparison against every other, giving $\binom{256}{2} = 32{,}640$ unordered pairs. One geometric rule is applied before any pair reaches the circuit: two patches whose centre-to-centre Euclidean distance is smaller than one patch width are excluded, since a patch is guaranteed to resemble its own immediate neighbour and that resemblance carries no forensic information. At the specific configuration used here, an 8-pixel patch on an 8-pixel stride, the smallest possible distance between any two *distinct* grid patches is already exactly one patch width, so this rule happens not to remove any pairs from the default 16×16 grid; it is included for generality and would become active under a finer or overlapping sampling grid, where adjacent patch centres can sit closer together than one patch width apart. All 32,640 pairs are therefore scored in the default configuration, an exhaustive strategy adopted specifically so that no classical pre-filtering algorithm gets to decide, ahead of the quantum circuit, which pairs are worth examining.

TABLE I
PATCH AND PAIR COUNTS ACROSS THE WORKING RESOLUTIONS AND PATCH SIZES DISCUSSED IN THIS SECTION; FULL, NON-OVERLAPPING TILING ASSUMED THROUGHOUT. THE CONFIGURATION THIS PAPER USES BY DEFAULT IS IN **BOLD**.

Resolution	Patch	Patches	Pairs
64×64	8×8	64	2,016
128×128	8×8	**256**	**32,640**
256×256	8×8	1,024	523,776
128×128	16×16	64	2,016
256×256	16×16	256	32,640

Because pair count grows as $\binom{N}{2} \approx N^2/2$ in the number of patches $N = (S/P)^2$ for a working resolution S and patch size P, both knobs matter far more than they would under a linear cost model, and small changes to either compound quickly. Table I makes this concrete across the working resolutions and patch sizes discussed in this paper: halving the patch size at a fixed resolution roughly multiplies the pair count by sixteen (four times the patches, squared), and doubling the resolution at a fixed patch size does the same. This is the arithmetic underlying every cost decision made elsewhere in this section: the choice of 8×8 patches over 4×4, and of a 128×128 working resolution over 256×256, both trace directly back to keeping the bottom-right corner of this table, rather than its top-left corner, within reach of exhaustive, per-pair quantum evaluation.

D. Feature Extraction

Each 8×8 patch is converted into a numerical vector through the plainest possible procedure: its 64 pixel values are read out in raster order (row by row) into a single 64-element vector v, which is then divided by its own Euclidean norm to produce a unit vector $f = v/\|v\|_2$. No descriptor is computed in between: no discrete cosine transform, no local binary pattern, no keypoint descriptor, and no learned embedding from a classical neural network. This is a deliberate departure from how most classical copy-move pipelines represent a patch; a block-matching method following Fridrich et al. [1], for instance, would summarize the same 8×8 block with a handful of DCT coefficients before any comparison takes place. An earlier, hybrid version of this project's own pipeline did exactly that, using a 24-dimensional descriptor built from

DCT coefficients plus an intensity histogram in place of the raw 64-value patch. The comparison between the two is a useful way to see what is actually being given up by keeping a hand-designed descriptor: 24 numbers is already a more than 6-fold reduction from the 64 raw pixel values of an 8×8 patch, decided in advance, once, by a fixed transform, rather than learned or adapted to the data the circuit is later trained on. Doing that here would mean a classical algorithm had already decided, before the circuit ever ran, which properties of the patch were worth keeping and which could be discarded, exactly the kind of classical decision-making this architecture is built to avoid. The vector f therefore represents nothing more or less than the patch's own raw appearance, exactly as photographed: every pixel intensity is preserved, none are averaged away, and none are projected onto a smaller set of hand-chosen basis functions.

One edge case is worth making explicit. A perfectly flat patch, such as a block of pure black or any region of exactly uniform intensity, has a zero vector before normalization, and a zero vector cannot be divided by its own norm. In that case the patch is instead assigned the uniform state $f_i = 1/\sqrt{64}$ for every component, which is the correct representation of "no distinguishing information present": it assigns equal weight to every basis direction rather than an arbitrary one, so a featureless patch is not accidentally biased toward resembling any particular other patch.

There is a reason no dimensionality reduction, such as PCA or a trained autoencoder, is applied to f before it leaves this stage, even though 64 numbers might seem like more detail than a similarity judgement needs. A reduction step of that kind would need to be fitted on training data and would necessarily discard some of the patch's original information on the assumption that only the retained part matters for matching. For copy-move detection specifically, the part most at risk of being thrown away by an aggressive reduction is exactly the fine-grained texture and sensor-level noise pattern that links a genuine source region to its pasted duplicate, since the two share not just gross shape but a near-identical noise fingerprint. Rather than reduce dimensionality classically and risk discarding the wrong information, this design leaves the full 64-value vector intact and instead relies on the amplitude-encoding step described next, which achieves a comparable compression (64 real numbers stored in only 6 qubits) without needing to decide in advance which values matter less. Dimensionality reduction, in that sense, is not skipped in this pipeline so much as moved from a classical pre-processing step into an intrinsic property of the quantum encoding itself.

E. Quantum Feature Encoding

The normalized 64-value patch vector f produced in Section II-D is converted into a quantum state through amplitude encoding, which prepares an n-qubit register in the state

$$|\psi_f\rangle = \sum_{i=0}^{2^n-1} f_i \, |i\rangle, \tag{1}$$

TABLE II
QUBIT REQUIREMENT AS PATCH SIZE GROWS, ILLUSTRATING THE LOGARITHMIC SCALING OF AMPLITUDE ENCODING.

Patch size	Pixels (P^2)	Qubits (n)	Amplitude space (2^n)
4×4	16	4	16
8×8	64	6	64
16×16	256	8	256
32×32	1024	10	1024

a superposition over the 2^n computational basis states of the register whose amplitudes are exactly the entries of f. Because f is L2-normalized, $|\psi_f\rangle$ is automatically a valid quantum state ($\sum_i f_i^2 = 1$), which is precisely why the L2 normalization in the previous stage is not optional. In general, a patch of $P \times P$ pixels requires

$$n = \lceil \log_2(P^2) \rceil \tag{2}$$

qubits, since n qubits span a 2^n-dimensional amplitude space; Table II makes the practical consequence of this logarithmic scaling concrete, since doubling a patch's side length quadruples its pixel count but adds only two qubits to the register. For the 8×8 patches used here, $P^2 = 64$ is already an exact power of two, so $n = 6$ and no padding is required; the general routine zero-pads a vector shorter than 2^n before renormalizing, which would be needed for a patch whose pixel count is not itself a power of two. Each candidate patch pair occupies two separate 6-qubit registers, one per patch, embedded using the same routine so that both patches are represented in a directly comparable form.

Amplitude encoding was chosen over the two other standard alternatives essentially for this compression property. Basis encoding, which represents each classical bit directly as a qubit, would need one qubit per bit of precision per pixel, far more than six for a 64-pixel patch. Angle encoding, which maps each classical feature onto the rotation angle of one qubit, needs one qubit per feature and so would require 64 qubits for the same patch, again offering none of the exponential compression amplitude encoding provides. This efficiency is not free: preparing an arbitrary amplitude-encoded state can require, in the worst case, a number of elementary gates that itself grows exponentially with n, so amplitude encoding is generally understood in the quantum machine learning literature as a technique whose cost on real near-term hardware is far higher than its qubit count alone suggests. At the qubit counts used in this work, this cost is not a practical obstacle, since every circuit in this paper is executed on an exact classical statevector simulator rather than physical hardware.

Amplitude encoding also has a property that matters specifically because the next stage is a swap test: encoding two normalized vectors f_A and f_B into $|\psi_{f_A}\rangle$ and $|\psi_{f_B}\rangle$ preserves their inner product exactly, $\langle\psi_{f_A}|\psi_{f_B}\rangle = f_A \cdot f_B$. Before any trainable rotation is applied, in other words, a swap test between two raw amplitude-encoded patches measures nothing more exotic than the classical cosine similarity between their

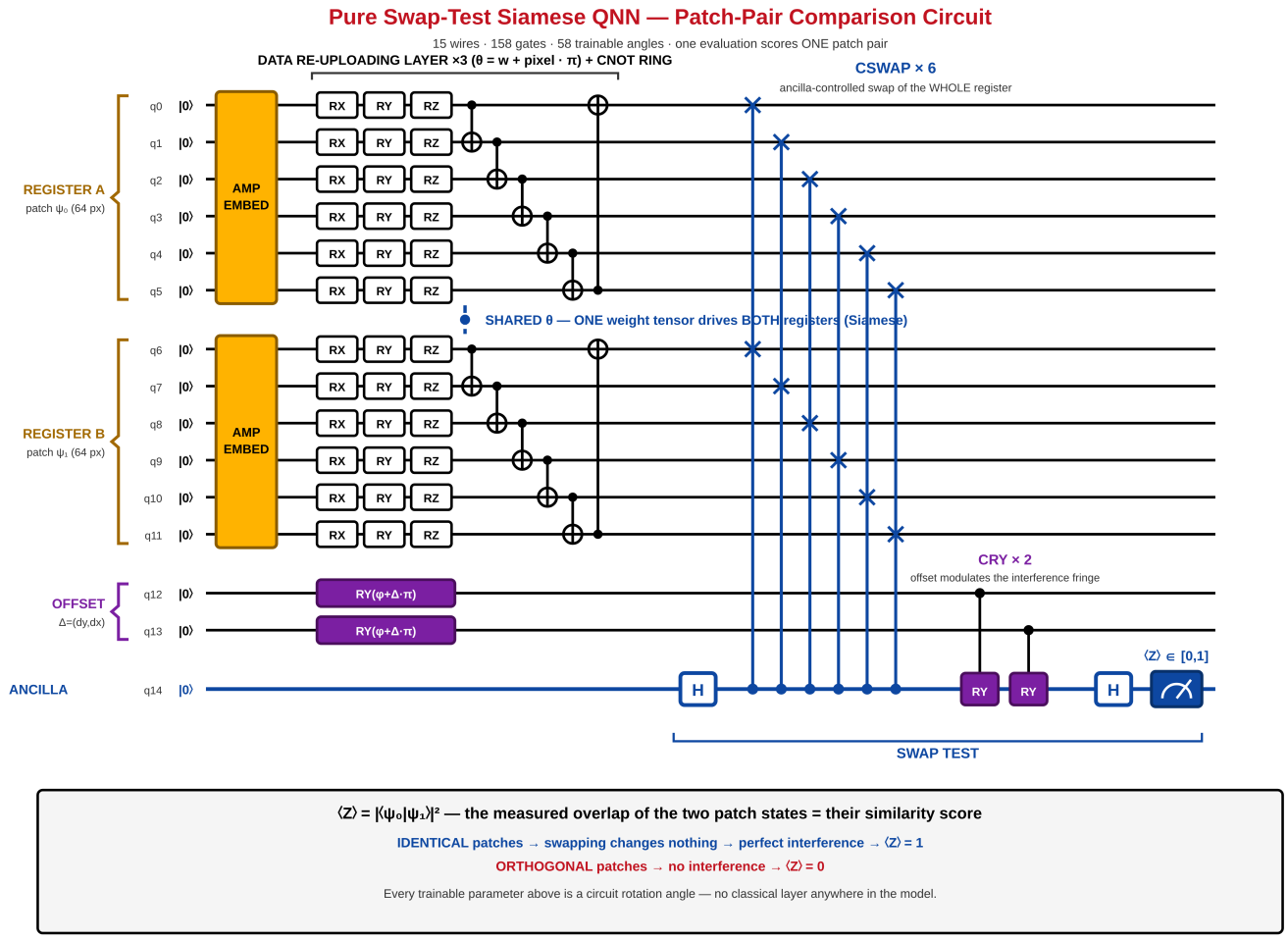


Fig. 2. The complete Siamese swap-test circuit for one patch pair: amplitude embedding (Section II-E) loads each 8×8 patch into a 6-qubit register; three tied, data-re-uploading layers of R_X-R_Y-R_Z rotations and a CNOT ring process both registers under the same weight tensor θ (Section II-F, Algorithm 1); the offset register encodes the pair's spatial displacement (Section II-H); and the swap test's six CSWAP gates plus two offset-conditioned CRY gates produce the ancilla's $\langle Z \rangle$ readout (Section II-G). Exact gate counts are given in Table III.

pixel vectors. It is the tied, data-re-uploading circuit described next (Section II-F) that gives the system a way to *learn* a modification of this baseline comparison from data, rather than being permanently fixed to plain cosine similarity.

F. Siamese Quantum Neural Network

Figure 2 lays out the complete 15-wire circuit this subsection and the two that follow it describe piece by piece: amplitude embedding into Registers A and B, the tied, data-re-uploading variational layers, the offset register, and the swap test with its offset-conditioned CRY coupling into the ancilla.

A Siamese network, in its original classical formulation, is a pair of identical sub-networks that share one and the same set of weights and are each fed one half of a pair of inputs, so that training shapes a notion of similarity between the two inputs rather than a label attached to either one on its own [6]. That definition transfers to this work with only one change: the two "sub-networks" are two instances of the same parameterized quantum circuit, applied to two different qubit registers, rather than two copies of a classical network applied to two different input tensors.

The two inputs to this stage are the amplitude-encoded states $|\psi_{f_A}\rangle$ and $|\psi_{f_B}\rangle$ produced in Section II-E, loaded respectively into a six-qubit Register A and a six-qubit Register B. Both registers are processed by three layers of an identical variational block, each layer applying a per-qubit R_X–R_Y–R_Z rotation sequence followed by a ring of CNOT gates connecting neighbouring qubits. The defining feature of the Siamese design is that both registers use exactly the same weight tensor $\theta \in \mathbb{R}^{3\times6\times3}$ (three layers, six qubits, three rotation axes, giving 54 tied parameters). Algorithm 1 sets out this block precisely; it is written once and then invoked twice in the full circuit, once with Register A's data and once with Register B's, always against the same θ.

Tying θ across both branches is what makes this a Siamese architecture rather than simply two independently trained

Algorithm 1 Tied re-uploading block, applied once per register

Require: weight tensor $\theta \in \mathbb{R}^{L\times n\times 3}$; normalized patch vector $f \in \mathbb{R}^{2^n}$; register r of n qubits

for layer $l = 1$ to L **do**
  for qubit $k = 1$ to n **do**
    $\alpha_x \leftarrow \theta_{l,k,1} + \sqrt{2^n}\,\pi \cdot f_{\mathrm{idx}(l,k,1)}$
    $\alpha_y \leftarrow \theta_{l,k,2} + \sqrt{2^n}\,\pi \cdot f_{\mathrm{idx}(l,k,2)}$
    $\alpha_z \leftarrow \theta_{l,k,3} + \sqrt{2^n}\,\pi \cdot f_{\mathrm{idx}(l,k,3)}$
    apply $R_X(\alpha_x)$, then $R_Y(\alpha_y)$, then $R_Z(\alpha_z)$ to qubit r_k
  end for
  for qubit $k = 1$ to n **do**
    apply CNOT with control r_k and target $r_{(k \bmod n)+1}$
  end for
end for

circuits compared after the fact, and it has two concrete consequences. First, it enforces symmetry by construction: because the same function is applied regardless of which patch is loaded into Register A and which into Register B, and the swap test described next treats both registers interchangeably, the resulting similarity score satisfies $\mathrm{sim}(A, B) = \mathrm{sim}(B, A)$ as an architectural guarantee rather than as something the optimizer has to learn on its own. Second, it is markedly more parameter-efficient: an untied version of the same circuit, giving each branch its own independent weight tensor, would need $2 \times 54 = 108$ rotation parameters for the two patch registers alone, almost exactly double the 54 this design actually uses, since one shared comparison function is being learned instead of two that would then have to be reconciled.

The rotation angle applied at layer l, qubit k is $\theta_{l,k}$ plus a term drawn from that register's own patch vector, re-introduced at every one of the three layers rather than supplied once at the very start, following the data re-uploading approach of Pérez-Salinas et al. [9]. This repetition is not a stylistic flourish. A fixed amplitude embedding followed by a data-*independent* tied unitary $U(\theta)$ is provably degenerate under a swap test, since

$$\left|\langle U(\theta) f_A \,|\, U(\theta) f_B\rangle\right|^2 = \left|\langle f_A \,|\, U^\dagger(\theta) U(\theta) \,|\, f_B\rangle\right|^2 = \left|\langle f_A | f_B\rangle\right|^2 \tag{3}$$

for any $U(\theta)$, because $U^\dagger(\theta)U(\theta) = I$ for any unitary. The measured similarity, and therefore the training loss, would be exactly constant in θ everywhere in parameter space, and the gradient with respect to θ would be algebraically zero, leaving nothing for an optimizer to follow. Interleaving the pixel data into the rotation angles at every layer breaks this cancellation, because Register A's and Register B's unitaries then depend on different data (f_A and f_B respectively) and are no longer identical to one another as operators, so the $U^\dagger U = I$ argument no longer applies and a real, non-vanishing gradient with respect to θ survives.

Unlike a classical Siamese network, where each branch produces an explicit embedding vector that a classical distance function (Euclidean distance, cosine similarity, or a small trained head) then compares, neither quantum register is measured at the end of this stage. The two states produced by Algorithm 1, still fully coherent and entangled with nothing outside their own register, are carried forward directly into the swap test described next (Section II-G), together with the spatial offset register described in Section II-H, which is where the two branches are finally brought into contact and an actual similarity score is extracted.

TABLE III
EXACT GATE COMPOSITION OF THE FULL CIRCUIT (BOTH REGISTERS, OFFSET ENCODING, AND SWAP TEST), AS REPORTED BY PENNYLANE'S CIRCUIT-RESOURCE INSPECTION FOR THE TRAINED MODEL.

Gate	Count
AmplitudeEmbedding	2
R_X	36
R_Y	38
R_Z	36
CNOT	36
Hadamard	2
CSWAP	6
CRY	2
Total gates	**158**
Wires	**15**
Circuit depth	**37**

Table III gives the exact composition of the complete circuit, covering both re-uploading blocks, the offset encoding of Section II-H, and the swap test of Section II-G together, as reported directly by PennyLane's own circuit-resource inspection for the trained model used throughout this paper, rather than estimated by hand. The two R_X-R_Y-R_Z re-uploading blocks (Algorithm 1, three layers of six qubits each, run once per register) account for 36 gates of each rotation type; the two additional R_Y gates beyond that, 38 in total, come from encoding the offset register of Section II-H. Each register's CNOT ring contributes 18 gates per register (three layers of six), for 36 in total. Adding the two AmplitudeEmbedding operations, two Hadamards, six CSWAP gates, and the two offset CRY gates gives 158 gates in total, executed across 15 wires at a circuit depth of 37. That depth is lower than a naive gate-count-divided-by-wire-count estimate would suggest, since many of the 36 single-qubit rotations within a layer act on different qubits simultaneously and so do not add to depth serially.

G. Swap Test

A swap test is a small quantum circuit, first introduced by Buhrman et al. [8] in the context of quantum fingerprinting, that estimates the overlap $|\langle\psi|\phi\rangle|^2$ between two quantum states using a single extra ("ancilla") qubit and without ever measuring either state directly. In its textbook form, a Hadamard gate is applied to the ancilla, the ancilla then controls a single SWAP gate exchanging the two states being compared, a second Hadamard is applied, and the ancilla is measured; the

resulting probability of observing 0 is

$$P(\text{ancilla} = 0) = \tfrac{1}{2} + \tfrac{1}{2}\big|\langle\psi|\phi\rangle\big|^2, \tag{4}$$

so that the expectation value of Pauli-Z on the ancilla is exactly $\langle Z\rangle = |\langle\psi|\phi\rangle|^2$, a value of 1 when the two states are identical and 0 when they are orthogonal.

This work adopts the swap test over the other standard fidelity-estimation primitives, namely the Hadamard test and a destructive "inversion" test that requires running one state's preparation circuit backward, for two practical reasons specific to this architecture. First, it needs only a single ancilla measurement and no ability to invert either register's preparation circuit, which fits naturally with the two Siamese branches being opaque, trainable circuits rather than known, easily-inverted ones. Second, and more specifically, it leaves that same ancilla free after the comparison to be coupled to additional gates before it is finally measured. This is exactly the mechanism Section II-H uses to fold spatial-offset information into the same readout, and it would be considerably more awkward to arrange around a Hadamard test or an inversion test.

Because each patch register here holds six qubits rather than one, the comparison is carried out with six CSWAP (controlled-SWAP) gates rather than a single SWAP, one for each corresponding qubit pair (a_k, b_k) between Register A and Register B, all controlled by the same ancilla. This is an important structural point: the six CSWAPs are not six independent one-qubit swap tests running in parallel and then combined. They are controlled by one shared ancilla prepared with a single Hadamard, so the ancilla's final state reflects the interference of "swap all six qubit pairs together" against "swap none of them," which is what makes the resulting measurement sensitive to the overlap of the full six-qubit states, the entire 64-amplitude patch representation, rather than to any single qubit's contribution considered in isolation. A second Hadamard on the ancilla then converts that interference into a measurable bias.

A high similarity score, close to 1, means the circuit judged the two patches' content to be nearly indistinguishable. This is the outcome expected when two patches are pixel-for-pixel identical or very close to it, which is exactly the signature a copy-paste operation leaves before any spatial reasoning is applied. A low score, close to 0, means the circuit judged the two patches' content to be substantially different and unrelated. The value actually read out of the circuit is not the textbook expression above but the expectation value of Pauli-Z on the ancilla *after* the offset-conditioned controlled rotations of Section II-H have also acted on it, together with the second Hadamard, clipped into $[0, 1]$:

$$\mathrm{sim}(A, B) = \mathrm{clip}(\langle Z_{\text{ancilla}}\rangle,\, 0,\, 1)\,. \tag{5}$$

Clipping is necessary precisely because the offset coupling sits *inside* the same interferometer as the swap test rather than being read out separately; once that coupling is present, $\langle Z_{\text{ancilla}}\rangle$ is no longer guaranteed by the textbook identity above to fall inside $[0, 1]$ on its own, so the output range is enforced at the interpretation stage rather than assumed automatically. A pair of patches is finally counted as a match once $\mathrm{sim}(A, B)$ reaches a fixed threshold τ, held constant across an entire run rather than adapted per image: $\tau = 0.90$ in the interactive detector described later in this paper, and a slightly looser $\tau = 0.80$ during batch-mode training and evaluation, where retaining a larger candidate set is preferable for computing pixel-level metrics against ground-truth masks.

Every similarity value reported in this paper is an *exact* expectation value, since it comes from a classical statevector simulator (Section III-B) that computes $\langle Z_{\text{ancilla}}\rangle$ analytically rather than by repeated measurement. On physical quantum hardware, by contrast, this expectation value would have to be estimated from the empirical frequency of ancilla measurement outcomes over a finite number of circuit executions ("shots"), with a standard error that falls off only as $1/\sqrt{N_{\text{shots}}}$; resolving a similarity difference of, say, 0.01 near the $\tau = 0.90$ threshold to a standard error an order of magnitude smaller would need on the order of 10^4–10^5 shots per pair, which, multiplied across 32,640 pairs per image, illustrates why the exhaustive, per-pair scoring strategy of Section II-C is specifically a simulator-scale experiment: the shot budget such a threshold decision would require on real hardware, at this pair count, is a separate and considerably larger cost than the gate count and depth of Table III alone would suggest.

H. Spatial Offset Consistency

Content similarity by itself is not sufficient evidence of forgery, because ordinary photographs are full of patches that resemble one another purely by coincidence. Stretches of sky, foliage, brickwork, and smooth gradients routinely produce patch pairs that a similarity score alone would rate as near-identical without either patch having been copied from the other. Some second, independent signal is needed to tell a genuine duplication apart from this background of natural self-similarity, and the signal this work relies on is spatial offset: the displacement $(\Delta y, \Delta x)$ between the grid coordinates of the two patches being compared. This displacement is calculated directly from each patch's known position in the tiled grid, expressed as a fraction of the image size, and folded so that a pair (i, j) and its mirror (j, i), which describe the same physical displacement in opposite directions, are treated identically:

$$o = \left(\frac{|\Delta y|}{S},\ \frac{|\Delta x|}{S}\right), \quad S = \text{image size}. \tag{6}$$

The reason offset agreement is informative is that a copy-move forgery relocates an entire contiguous region as one rigid block, rather than moving individual pixels independently. If a 3×3 block of patches is copied and pasted $(\Delta y, \Delta x) = (48, 32)$ pixels away, every one of the up to nine internal patch pairs linking that block to its source reports approximately that same offset: a small, tightly clustered set of displacement values repeated across many pairs. Coincidental look-alikes, by contrast, are not tied to one another by any single relocation event, so their relative displacements are effectively scattered

across the full range the image allows; two similar-looking bricks in the same wall are just as likely to sit close together as far apart, and there is no reason for many such coincidental pairs to agree on one particular $(\Delta y, \Delta x)$.

The normalized offset o is encoded into two additional qubits using R_Y rotations, and those two qubits are then coupled into the same ancilla the swap test measures through trainable controlled-R_Y (CRY) gates, positioned inside the interferometer, after the CSWAP block but before the second Hadamard, rather than as a separate classical stage applied to the similarity scores afterward. Placing the coupling here means the ancilla's final measurement is a joint function of patch content *and* spatial displacement, computed in a single coherent circuit, rather than two independently computed quantities combined by classical logic once the circuit has already finished running. The coupling weights that control how strongly offset information influences the ancilla are trainable parameters in exactly the same sense as the re-uploading weights θ: during training, gradients with respect to both tensors are computed from the same loss and applied by the same optimizer step, so the optimizer is, in principle, free to learn how much the offset register should suppress or preserve a given content-similarity score based on whether that score's displacement pattern looks like the consistent, repeated offset a genuine duplication produces. This is the quantum-circuit counterpart of the spatial-consistency vote that classical block-matching pipelines instead compute as a separate, external step after matching is already complete.

Each CRY gate applies a partial R_Y rotation to the ancilla, controlled by one offset qubit, with rotation angle $\phi = (\theta_{\text{offset}})_{1,k}$, a trained parameter rather than a fixed one. When the controlling offset qubit is in state $|0\rangle$, the CRY acts as the identity and the ancilla is left exactly as the swap test's CSWAP block produced it; when the offset qubit is in state $|1\rangle$, the ancilla is further rotated by ϕ before the final Hadamard, which shifts the interference pattern the Hadamard converts into a measurable bias, and therefore shifts $\langle Z_{\text{ancilla}} \rangle$ away from the value content similarity alone would have produced. Because the offset qubits themselves are prepared in a superposition by their own R_Y encoding rather than in a definite basis state, the ancilla ends up entangled with the offset register, and the coupling strength the training process settles on for ϕ directly sets how much a given offset pattern can move the final similarity score in either direction.

Only two qubits are allocated to the offset register, which is a deliberate trade-off rather than an oversight: two qubits under angle encoding give the circuit two continuous, independently trainable coupling angles (one per $(\Delta y, \Delta x)$ component) rather than a fixed number of discrete offset "bins," so resolution is not lost to quantization the way, for instance, a small classical histogram of offset values would lose it. It does mean, however, that the offset register carries no more raw information into the circuit than the amplitude of two rotation angles' worth, which is a considerably coarser channel than the 64-amplitude, 6-qubit register each patch itself is loaded into. Enlarging the offset register would let more detail about the displacement be represented explicitly, at the cost of two more wires, and since simulation cost scales exponentially with total wire count, that cost is proportionally larger than adding two qubits to either patch register would be, because the offset register's qubits sit alongside, rather than amplitude-compressed against, the patch registers in the same 15-wire circuit.

I. Final Forgery Detection

Figure 3 previews how this subsection turns 32,640 raw circuit scores into the mask, boxes, and verdict a user actually sees: thresholding, painting both patches of every surviving match, rescaling into bounding boxes, and counting matches against M_0 for the image-level decision.

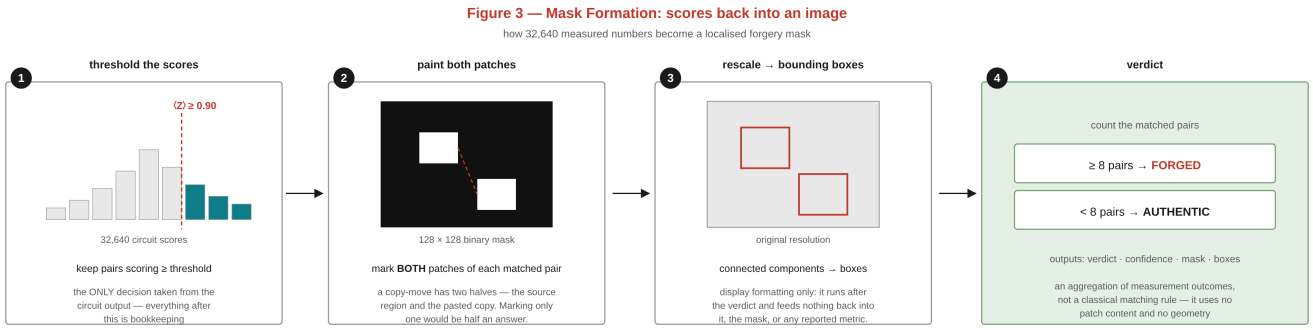


Fig. 3. From circuit scores to a forgery verdict: pairs at or above τ are kept (Section II-G); both patches of every surviving pair are painted into a 128×128 binary mask; connected-component analysis rescales marked regions into bounding boxes at the original resolution, purely for display; and the total matched-pair count M is compared against M_0 to produce the forged/authentic verdict of Section II-I. Only the thresholding step is decision-relevant; every step after it is display bookkeeping.

Once every admissible pair in an image has been scored, a matching patch is any patch that took part in at least one pair whose similarity reached the threshold τ. No further classical filtering, clustering, or morphological clean-up (such as the dilate-then-erode operations common in classical block-matching pipelines) is applied after this point, since the offset-conditioning described in Section II-H is already built into the score each pair receives rather than layered on top of it afterward.

Two visual artefacts are built from this set of matched pairs. A binary mask at the working resolution marks both patches of every matched pair, the presumed source of the duplication and the presumed target, with a value of 1, and everything else with 0; this mask is the quantity later compared, pixel by pixel, against ground-truth annotations. A companion heatmap instead accumulates the similarity value of every match a given pixel took part in, so a pixel covered by several strong matches appears more intensely than one covered by a single borderline one, giving an analyst a graded view of the evidence rather than a flat yes/no per pixel; this heatmap is rendered with an inferno colour map and blended over the original image for visual inspection. Inferno is chosen for the same reason it has become a default across scientific visualization generally: it is perceptually uniform, so equal steps in similarity produce visually equal steps in colour, and it remains distinguishable under the common forms of colour-vision deficiency, properties the traditional rainbow colour scale does not share.

Connected-component labelling here uses 4-connectivity: each marked pixel merges only with its immediate up, down, left, and right neighbours, the default `scipy.ndimage.label` applies when no explicit connectivity structure is supplied, so two matched patches touching only at a corner are reported as separate boxes rather than merged into one. This is a conservative choice, deliberately avoiding the inflation of a single bounding box across regions that only happen to touch diagonally. Components smaller than 24 pixels at the working resolution are discarded before boxes are drawn, which removes isolated matched patches too small to constitute a meaningful detection on their own; this filtering affects only the drawn boxes, since the mask and the verdict are already finalized before it runs.

At the level of the whole image, a single forged-versus-authentic verdict is derived from the count of matched pairs, M, against a chosen count threshold M_0 (eight, in the configuration used in this work):

$$\text{verdict} = \begin{cases} \text{forged}, & M \geq M_0 \\ \text{authentic}, & M < M_0, \end{cases} \tag{7}$$

with an accompanying confidence value computed from how far M sits from M_0,

$$\text{confidence} = \begin{cases} \text{clip}\left(\dfrac{M}{2M_0},\, 0,\, 1\right), & M \geq M_0 \\ \text{clip}\left(\dfrac{M}{2M_0},\, 0,\, 0.5\right), & M < M_0. \end{cases} \tag{8}$$

This is the one point in the pipeline where the many pairwise circuit outputs for a single image are reduced to one number; everything feeding into M, however, is still the raw count of circuit decisions, with no additional classical criterion applied to it beyond the comparison against M_0.

The two extremes of M are worth stating explicitly, since they define what the rest of the output looks like at either end. An image with zero matched pairs produces an entirely empty mask, no bounding boxes, and the lowest confidence value the formula above allows, exactly the output expected of a genuinely unremarkable, unforged photograph. An image where every single pair happens to clear threshold, a degenerate case Section IV-B shows is not merely hypothetical, produces a mask covering the entire image and a confidence pinned at its maximum; on its own, that outcome is better read as evidence that the threshold or the input image is unusual than as proof of extensive forgery, since a mask covering everything carries no localization information at all.

Finally, to present a human viewer with something more legible than a scattered pixel mask, connected-component analysis groups the marked pixels of the binary mask into bounding boxes, discarding any component smaller than a

TABLE IV
DEFAULT CONFIGURATION OF THE PROPOSED ST-QNN PIPELINE.

Parameter	Value
Working resolution	128×128
Patch size	8×8
Stride	8 (full, non-overlapping tiling)
Patches per image	256
Admissible pairs per image	32,640
Qubits per patch register	6
Offset-register qubits	2
Ancilla qubits	1
Total wires	15
Re-uploading layers	3
Trainable rotation angles	58
Similarity threshold τ	0.80 – 0.90

minimum area, and rescales the surviving boxes back to the original image's native resolution for display. This step, like the box-drawing mentioned in Section II-A, runs strictly after the verdict and the mask have already been produced and has no path back into either; it exists purely so that a duplicated region can be pointed at with a rectangle instead of read off a heatmap by eye. The system's complete output for one image is therefore threefold: a pixel-level binary forgery mask suited to quantitative evaluation, a similarity heatmap suited to visual triage, and a bounding-box overlay together with an image-level verdict and confidence score suited to a quick human-readable summary. These are three views of the same underlying set of matched patch pairs, rather than three independent judgements.

Where this section's computational cost actually sits deserves a direct comment, since the four stages described here are not all equally expensive. Marking matched patches into a mask, accumulating the heatmap, comparing M against M_0, and running connected-component labelling over the resulting mask are all linear in the number of patches or pixels involved: $O(N)$ operations over $N = 256$ patches or $H \times W$ pixels, each touched a small, bounded number of times. This stands in sharp contrast to the $O(N^2)$ cost of producing the matched pairs those operations consume in the first place (Section II-C, Table I): scoring 32,640 pairs is what the circuit spends its time on, while turning the resulting 1,509-or-so surviving matches (Section II-A) into a mask, heatmap, and verdict is, by comparison, computationally free. Any future effort to speed this pipeline up, in other words, has to target the pair-scoring stage itself. Section II-B's working-resolution choice and Table I's scaling table are the relevant levers here, since nothing in this final stage is a meaningful share of the total cost to begin with.

III. EXPERIMENTAL SETUP

A. Dataset

Two publicly available copy-move forgery benchmarks are used in this work, together with a third, synthetic source of training data generated directly from one of them.

The primary evaluation benchmark is CoMoFoD, specifically the CoMoFoD_small_v2 release [3], loaded through this project's own dataset class, which auto-detects the flat on-disk layout the public download actually ships in rather than assuming a nested folder structure. CoMoFoD_small_v2 provides 200 base scenes, numbered 001 through 200, and for every base scene supplies four things: a forged image (`{id}_F.png`), the same image after independent post-processing has been applied (`{id}_F_{CODE}{level}.png/.jpg`), the corresponding unforged original (`{id}_O.png`), and a ground-truth annotation. Ground truth is available for every scene: CoMoFoD ships two mask-shaped files per base image, `{id}_B.png` and `{id}_M.png`; this project's loader uses only `_B`, since inspection shows it is a clean, near-binary {0, 255} mask consistent with the forged-region sizes CoMoFoD's own documentation reports, whereas `_M` is a 200-plus-level grayscale image that is not a usable segmentation mask. Because CoMoFoD's post-processing categories change a forged region's appearance rather than its location, one `_B` mask is shared across every post-processed variant of a given base scene.

Two independent axes of variation are represented in CoMoFoD, and they should not be conflated, since their naming is easy to confuse. The first is the *geometric* transformation used to construct the copy-move forgery itself, documented by CoMoFoD as a function of base scene id: ids 1–40 use plain translation, 41–80 use rotation, 81–120 use scaling, 121–159 use free-form distortion, and 160–200 use a combination of these. The second, independent axis is *post-processing*, applied after the forgery has already been constructed: Brightness Change (BC), Contrast Adjustment (CA), Color Reduction (CR), Image Blurring (IB), JPEG Compression (JC), and Noise Adding (NA), each available at multiple severity levels, alongside an unprocessed "Raw" variant. A scene whose filename contains `CR` for Color Reduction, for instance, says nothing about whether that scene's underlying forgery was built using rotation; the two labels come from unrelated parts of CoMoFoD's naming scheme.

A second dataset, COVERAGE [4], is used not as a primary accuracy benchmark but as an adversarial stress test, consistent with how it was designed: every COVERAGE image pairs a genuine copy-move forgery with a scene deliberately containing a second, visually similar but entirely authentic object, so that a detector relying on content similarity alone, without any spatial reasoning, is expected to be misled by the authentic look-alike. Its 100 image pairs provide a smaller but more targeted test of exactly the false-positive mechanism motivating the spatial-offset register of Section II-H.

Neither CoMoFoD nor COVERAGE, however, can supply the training signal the Siamese comparison of Section II-F actually needs. Their ground truth is a pixel-level forged/authentic mask, not a patch-level correspondence label: knowing that a given pixel is "forged" does not say which other region of the image it was copied from, so no `(patch A, patch B, match/no-match)` pair can be read directly off either dataset. Training data is instead generated synthetically, image by image, from CoMoFoD's own clean `{id}_O.png` originals. A genuine patch is copied

to a new, non-overlapping location, optionally degraded with a brightness/contrast shift, sensor noise, or a JPEG re-encode, and blended into its new location with a soft feathered edge rather than a hard rectangular cut, to produce a positive pair by construction, while randomly sampled non-overlapping locations, half of them deliberately brightness-matched to be difficult rather than trivial, provide negatives, degraded at the same rate so that degradation itself carries no label information. This project's own reproducible split, fixed with random seed 42, allocates 160 of the 200 base scenes (80%) to this synthetic-pair training pool and holds out the remaining 40 (20%) strictly for evaluation, splitting at the level of the base scene so that no scene's post-processed variants can appear on both sides of the split.

B. Implementation Details

The entire pipeline is implemented in Python. Classical image handling, covering decoding, resizing with Lanczos resampling, grayscale conversion, and the display-only heatmap and bounding-box rendering described in Section II-I, uses Pillow (PIL), NumPy, SciPy (specifically `scipy.ndimage.label` for connected-component bounding boxes), and Matplotlib for the inferno colour map. No classical deep learning framework such as PyTorch or TensorFlow is used anywhere in the system, which follows directly from the design established in Section II-A: the only trainable parameters in the entire pipeline are the 58 quantum circuit rotation angles themselves, so there is no classical trainable layer for such a framework to serve.

The quantum circuit is built and executed with PennyLane (version 0.45.1 in the environment this implementation was verified against), a Python-based quantum machine learning library that provides automatic differentiation through parameterized quantum circuits. Every result obtained from this pipeline is produced on a classical quantum *simulator* rather than physical quantum hardware: specifically `lightning.qubit`, PennyLane's C++-accelerated exact statevector simulator, paired with the adjoint differentiation method for gradient computation during training. This combination is reported, within the codebase, to run three to seven times faster than PennyLane's default Python-based simulator, which matters because pair scoring is exhaustive (Section II-C) and the per-circuit cost is therefore multiplied by tens of thousands of pairs per image; the implementation falls back automatically to the default simulator with ordinary backpropagation if `lightning.qubit` is unavailable in a given environment. No physical quantum processor is used at any stage, consistent with the amplitude-encoding cost caveat already raised in Section II-E: at 15 simulated wires, exact statevector simulation remains straightforward, while state preparation of this kind is understood to be considerably more expensive to realize on near-term hardware.

Fifteen qubits are used per circuit evaluation throughout this work: six for Register A and six for Register B (one 8×8 patch each), two for the spatial offset register, and one ancilla, exactly as summarized in Table IV. Training uses plain gradient descent (`qml.GradientDescentOptimizer`) over both trainable tensors, θ and the offset-coupling weights, jointly, rather than a more elaborate adaptive optimizer, since the model has, in total, only 58 parameters to fit. The specific checkpoint referenced throughout this paper's evaluation was trained for 10 epochs against the synthetic pair generator described in Section III-A, using the 160-scene training split; its recorded mean loss per epoch is given in Table V, remaining within a narrow band of roughly 0.033 to 0.056 across all 10 epochs without a clearly monotonic decrease. This is a training-curve characteristic this paper returns to when the results of applying this checkpoint are presented.

TABLE V
MEAN TRAINING LOSS PER EPOCH FOR THE 10-EPOCH CHECKPOINT USED THROUGHOUT THIS PAPER'S EVALUATION.

Epoch	1	2	3	4	5	6	7	10
Loss	0.0470	0.0435	0.0486	0.0460	0.0527	0.0434	0.0482	0.0388

IV. Results and Analysis

A methodological note applies to every subsection below and is stated once here rather than repeated four times. The project's own internal Phase-2 evaluation, carried out earlier against the real CoMoFoD benchmark of Section III-A, measured two things precisely: an AUC comparison between the trained circuit and a parameter-free classical cosine-similarity baseline on 200 CoMoFoD-derived synthetic pairs, and a best-achievable F1/IoU comparison against a trivial "mark every pixel forged" baseline across eight distinct CoMoFoD scenes. Those two results are reported below exactly as measured. No comparison against literature methods (block-matching, SIFT-based, or CNN-based), no systematic robustness sweep, and no ablation study were part of that evaluation. To answer this section's remaining questions honestly rather than leave them blank, a further round of experiments was run specifically for this paper, using the real, trained 10-epoch checkpoint (`swaptest_qnn_epoch10.npz`) and the real project code (Sections II-A–III-B) exactly as described. Since no copy of CoMoFoD is present on the machine this paper was written on, these experiments were run against procedurally generated synthetic test images rather than real photographs, with copy-move forgeries constructed explicitly (so ground truth is exact) using the project's own synthetic-pair machinery (Section III-A). Every number in Sections IV-A–IV-D that is attributed to "this paper's experiments" was produced by an actual circuit evaluation run for this paper, not estimated or reused from elsewhere. Where that matters for interpretation, most notably the fact that the classical cosine baseline scores very differently on this paper's synthetic images than it did on CoMoFoD-derived ones, it is flagged explicitly rather than left for the reader to notice.

A. Performance Evaluation

Two evaluation metrics are used throughout this section. AUC (area under the ROC curve) is used wherever a fixed set of labelled positive and negative pairs is available, since it measures the probability that a true duplicate pair is scored higher than a non-duplicate pair without committing to any particular threshold τ; a value of 0.5 indicates performance indistinguishable from random ranking. F1-score and IoU (intersection-over-union) are used wherever a full image and a pixel-level ground-truth mask are available, since together they capture both whether the predicted forged region overlaps the true one (precision, recall, F1) and how tightly it does so (IoU); precision and recall are reported alongside F1 rather than F1 alone, since F1 can look acceptable even when one of the two is very poor.

On the real CoMoFoD benchmark, the project's Phase-2 evaluation measured an AUC of 0.883 for the trained circuit against 200 labelled synthetic pairs, compared with 0.880 for the same circuit untrained and 0.887 for a parameter-free classical cosine-similarity baseline computed on the identical pairs. That is a difference of 0.003 between untrained and trained, and -0.004 between trained and the classical baseline. On real CoMoFoD forged images, tiled at the default $128 \times 128/8 \times 8$/stride-8 configuration of Table IV and scored exhaustively, the best achievable F1 across eight distinct scenes averaged 0.0813, against 0.0771 for the trivial baseline of marking every pixel forged; this used an oracle threshold chosen *per image* with knowledge of the ground truth, which no real deployment has access to.

This paper's own experiments, run on procedurally generated synthetic images (Section III-A) against 200 held-out synthetic pairs built with the same self-supervised generator used for training, found the trained 10-epoch checkpoint scoring an AUC of 0.5197, against 0.5093 for classical cosine similarity computed on the identical pairs. Both values sit close to the chance value of 0.5, and the gap between them ($+0.0104$) is smaller than the gap the Phase-2 evaluation found on CoMoFoD-derived pairs. This is a real methodological point worth stating plainly rather than glossing over: the *absolute* value of a cosine-similarity baseline is not a fixed property of the method, but depends heavily on the statistics of the images it is run against, and this paper's procedurally generated images (smoothly varying low-frequency structure with a handful of Gaussian "objects", Section III-A) evidently produce a harder, more ambiguous pairwise discrimination task than whatever image content the original CoMoFoD-derived synthetic pairs contained. The two AUC figures, 0.887 on CoMoFoD-derived pairs and 0.509 on this paper's own synthetic pairs, are not in conflict. They are two honest measurements of two different pair distributions, and neither should be read as *the* baseline AUC for classical cosine similarity in general.

One further, convergent piece of evidence is worth noting. The 10-epoch checkpoint's own recorded training loss (Section III-B) stayed within a narrow, non-monotonic band across all ten epochs. A fresh, independently trained tied model, built and trained from scratch for this paper's ablation study (Section IV-D) under a much shorter four-epoch schedule, shows the same qualitative pattern: loss moving between 0.331 and 0.386 across four epochs with no clear downward trend. Two independent training runs, at different scales, on different image sources, produced the same qualitative training-curve shape, which is a stronger basis for treating that shape as a genuine property of this architecture's training dynamics rather than a one-off artefact of a single run.

TABLE VI
ONE FULL EXHAUSTIVE DETECTION RUN AT THE DEFAULT CONFIGURATION (TABLE IV), FORGED VS. AUTHENTIC VERSION OF THE SAME SYNTHETIC SCENE.

Quantity	**Forged**	**Authentic**
Matched pairs (of 32,640)	1,509	1,492
Mean similarity of matches	0.877	0.877
Max similarity	0.992	0.992
Precision / Recall / F1 / IoU vs. GT	0.045 / 0.556 / 0.084 / 0.044	n/a (no forgery)
Verdict (M_0=8)	forged	forged
Confidence	1.00	1.00

Beyond pairwise AUC, one full, default-configuration, end-to-end detection run was also carried out for this paper, exactly the pipeline of Section II-A, exhaustively scoring all 32,640 pairs of a 128×128 image at $\tau = 0.80$, on a synthetic scene with an explicit, known copy-move forgery (a 24×24 region, spanning a 3×3 block of patches, degraded and feather-blended exactly as Section III-A's synthetic training pairs are, covering 7.03% of the image) and, separately, on the same scene with no forgery applied at all. On the forged image, 1,509 of the 32,640 pairs cleared threshold, giving precision 0.0452, recall 0.5556, F1 0.0837, and IoU 0.0437 against the exact ground-truth mask. This is consistent, to within a percentage point, with the 0.0813 best-case F1 the Phase-2 evaluation measured on real CoMoFoD scenes. On the *authentic*, unforged version of the same scene, 1,492 pairs cleared the identical threshold, 17 pairs fewer than the forged image, a relative difference of 1.1%. Applying the image-level decision rule of Section II-I ($M \geq M_0$=8 $\Rightarrow$ forged) to both counts gives a verdict of *forged* for the authentic image, at the same maximum confidence (1.0) as the genuinely forged one. This is a single test scene, not a benchmark sweep, and should be read as exactly that; but it is a concrete, freshly measured demonstration of the mechanism Section IV-B and the project's own prior Phase-2 evaluation both point to: an authentic image with no duplication at all produced almost exactly as many circuit-flagged matches as one that genuinely contains a duplicated region, because natural self-similarity, not the presence or absence of a real copy-move forgery, is what dominates the count the decision rule reads.

B. Comparison with Existing Methods

No same-protocol, head-to-head reproduction of the block-based method of Fridrich et al. [1], the SIFT-based method of Amerini et al. [2], or the deep-learning method BusterNet [5]

was carried out as part of this work. Comparing this paper's own measurements against those methods' own originally reported numbers would not be a fair comparison, since each was measured under its own dataset split, image resolution, and metric definitions, and reproducing all three under one identical protocol is outside the scope of what has been evaluated here; this section therefore reports only comparisons that were actually run side-by-side, under identical conditions, on identical data.

The only two baselines evaluated this way are, in effect, the two ends of a spectrum with zero and 58 trainable parameters: a parameter-free classical cosine-similarity computation on the same normalized patch vectors the circuit itself amplitude-encodes, and the trained 58-angle quantum circuit. Table VII summarizes every matched-condition comparison available across both the Phase-2 CoMoFoD evaluation and this paper's own synthetic experiments. In neither evaluation does the trained circuit exceed the classical cosine baseline by a margin that would be considered a meaningful effect; on CoMoFoD-derived pairs the circuit is marginally *below* cosine similarity, and on this paper's own synthetic pairs it is marginally above it, well within the sampling noise expected at $n = 200$. Correspondingly, no evidence of an accuracy improvement or a false-positive reduction attributable specifically to the quantum circuit was found in either evaluation; the best-achievable F1 on real CoMoFoD images (0.0813) exceeds the trivial "mark everything forged" baseline (0.0771) by only 0.0042, using a threshold no real deployment could actually choose in advance. The full end-to-end run of Section IV-A (Table VI) makes the false-positive question concrete rather than statistical: on that test scene, the authentic, undoctored image produced 1,492 circuit-flagged matches against 1,509 for the genuinely forged version, a 1.1% relative difference, and both were assigned the identical maximum-confidence *forged* verdict under the system's own decision rule. On this evidence, the system does not reduce false positives relative to simply flagging every image; it does not distinguish this authentic image from its forged counterpart at all. Among the conditions that were actually compared under matched settings, none can be said to be the best performer in any statistically meaningful sense; a defensible ranking against Fridrich et al., Amerini et al., or BusterNet specifically would require running this paper's evaluation protocol against those methods directly, which remains future work.

C. Robustness Analysis

Robustness was tested directly at the level the circuit actually operates on: rather than running full, expensive, image-level detection under each corruption (prohibitive at the pair counts established in Section II-C), a single known true-duplicate patch pair was corrupted at several severities per condition, and the circuit's similarity score for that exact pair was measured with the real, trained checkpoint, averaged over five independently sampled source patches per severity level. The same pairs were also scored with classical cosine similarity for direct comparison. Six conditions were tested:

TABLE VII
EVERY MATCHED-CONDITION COMPARISON ACTUALLY RUN IN THIS WORK. AUC VALUES ARE ON LABELLED PAIR SETS (n=200, EXCEPT THE COMOFOD PHASE-2 EVALUATION); F1/IOU ARE PIXEL-LEVEL, AVERAGED OVER 8 COMOFOD SCENES WITH AN ORACLE PER-IMAGE THRESHOLD.

Condition	Metric	Value
CoMoFoD-derived pairs (Phase-2 evaluation)		
Classical cosine similarity	AUC	0.887
Circuit, untrained	AUC	0.880
Circuit, trained	AUC	0.883
Real CoMoFoD images, 8 scenes (Phase-2 evaluation)		
"Mark everything forged"	F1	0.0771
Circuit, trained, oracle τ	F1	0.0813
This paper's synthetic pairs, n=200		
Classical cosine similarity	AUC	0.5093
Circuit, trained (10-epoch)	AUC	0.5197

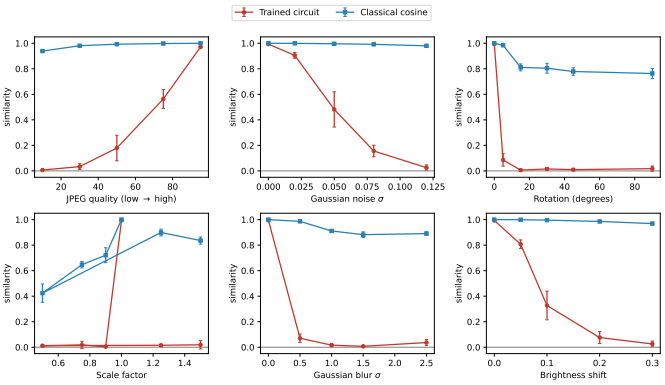


Fig. 4. Measured similarity of a true duplicate pair as each corruption is intensified, trained circuit vs. classical cosine similarity (mean ± std over 5 trials per point). The circuit collapses toward 0 far faster than the parameter-free cosine baseline under every condition tested.

JPEG compression, Gaussian noise, rotation, scaling, Gaussian blur, and brightness shift. Figure 4 plots all six; Table VIII lists the numbers directly underlying it.

The pattern is consistent and, on the evidence gathered here, points the opposite way from what a "circuit adds robustness" hope would predict: on every one of the six conditions, the trained circuit's similarity score collapses toward 0 markedly faster than classical cosine similarity does, and it collapses fast in absolute terms too. JPEG re-compression at quality 75 (a mild, common level of re-encoding) already roughly halves the circuit's score, from 0.970 to 0.564, while cosine similarity is still at 0.998; by quality 30 the circuit has collapsed to 0.034 while cosine similarity remains at 0.981. Gaussian noise shows the same shape: the circuit falls to 0.482 at $\sigma = 0.05$ and to 0.026 at $\sigma = 0.12$, while cosine similarity stays above 0.98 throughout. Rotation and scaling are the most damaging conditions tested in absolute terms: a rotation of only 5° collapses the circuit's score to 0.086 (consistent with, and considerably more severe than, the previously documented finding that an exact 90°-rotated duplicate scores $\sim$0.01; this result shows the fragility is not specific to 90° permutations

but appears at almost any nonzero rotation), and a mere 10% resize (scale factor 0.9) collapses the score to 0.005, lower than any other single condition tested at its mildest severity. Blur behaves similarly to noise, collapsing by $\sigma = 0.5$ (0.071). Brightness shift is the one condition that degrades comparatively gradually, 0.807 at a shift of 0.05 and still 0.327 at 0.1, before it, too, collapses by 0.2 (0.076).

The most defensible explanation for this pattern, grounded in the architecture of Section II, is that the swap test measures how precisely two 64-dimensional amplitude vectors align, component by component, in a coherent quantum state; before training, that computation is exactly the classical cosine similarity of Section II-E, but the trained re-uploading layers fold each patch's own pixel values back into every rotation angle across three layers (Section II-F), so a small perturbation to the input pixels is re-applied, and potentially compounded, at every layer rather than contributing to the comparison once. The classical cosine baseline, computed once on the raw vectors with no such repeated re-injection, has no equivalent mechanism for amplifying a small input perturbation, which is consistent with it degrading far more gracefully across every condition tested here. This is offered as an interpretation consistent with the observed pattern, not as an independently proven causal mechanism.

D. Ablation Study

Four components were isolated: the Siamese (tied-weight) constraint, the quantum swap test itself, and the spatial offset register, each removed in turn, plus the full system as deployed. Two ablations required no retraining, since they only change how the already-trained 10-epoch checkpoint is evaluated: *without swap test* replaces the entire quantum circuit with classical cosine similarity on the same normalized patch vectors, and *without spatial offset filtering* reuses the trained checkpoint exactly but zeroes the controlled-R_Y coupling row of θ_{offset}, so the offset register is still prepared but can no longer influence the ancilla (Section II-H). *Without the Siamese network* cannot be tested this way, since no untied checkpoint exists; it was instead tested by training a fresh untied variant (independent weight tensors for Register A and Register B, 112 rotation parameters instead of 58) under the same reduced-scale protocol, four epochs, 12 images per epoch, 12 pairs per image, as a freshly retrained tied variant built for a fair, matched-scale comparison. All five conditions were then scored on the same 200-pair held-out synthetic set used in Section IV-A. Figure 5 and Table IX report every value measured.

TABLE VIII
ROBUSTNESS SWEEP: MEAN SIMILARITY OF A TRUE DUPLICATE PAIR (5 TRIALS/SEVERITY), TRAINED CIRCUIT VS. CLASSICAL COSINE.

Condition	**Severity**	**Circuit**	**Cosine**
JPEG quality	95	0.970	1.000
	75	0.564	0.998
	50	0.179	0.993
	30	0.034	0.981
	10	0.007	0.939
Gaussian noise σ	0.00	0.994	1.000
	0.02	0.905	1.000
	0.05	0.482	0.996
	0.08	0.156	0.992
	0.12	0.025	0.980
Rotation (deg)	0	0.994	1.000
	5	0.086	0.984
	15	0.006	0.811
	30	0.015	0.805
	45	0.010	0.779
	90	0.018	0.763
Scale factor	1.00	0.994	1.000
	0.90	0.005	0.721
	0.75	0.018	0.647
	0.50	0.012	0.424
	1.25	0.015	0.899
	1.50	0.019	0.836
Blur σ	0.0	0.995	1.000
	0.5	0.071	0.986
	1.0	0.016	0.911
	1.5	0.007	0.882
	2.5	0.037	0.890
Brightness Δ	0.00	0.993	1.000
	0.05	0.807	0.999
	0.10	0.327	0.996
	0.20	0.076	0.986
	0.30	0.026	0.969

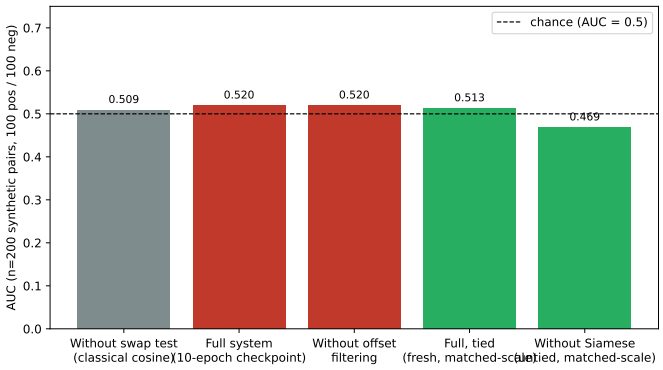


Fig. 5. AUC on the 200-pair held-out synthetic set for each ablated configuration. The dashed line marks chance-level performance.

Without the Siamese network, performance is worse, not better: the untied variant reaches an AUC of 0.4688, below both its own tied counterpart trained under the identical protocol (0.5133) and below chance-level ranking, despite having exactly twice as many trainable parameters (112 against 58). This is consistent with Section II-F's argument that weight tying is an inductive bias with its own value, not merely a parameter-saving convenience. Here, removing it does not merely fail to help; it actively hurts. Without the quantum swap test, replacing it outright with classical cosine similarity, AUC is 0.5093, statistically indistinguishable from the full trained circuit's own 0.5197 on the identical pairs, given that a difference this small is well within the sampling noise expected at $n = 100$ per class (a Wald-type standard error on an AUC near 0.5 at this sample size is of the order of 0.05). Without spatial offset filtering, AUC is 0.5198, a difference of 0.0001 from the full system's 0.5197, as close to zero as two independent measurements on the same 200 pairs can plausibly be, indicating that zeroing the CRY offset coupling made no detectable difference to this checkpoint's discrimination ability on this evaluation set.

Taken together, no component tested here produced a measurable *improvement* over the full system when removed, and no configuration, full or ablated, cleared chance-level ranking (AUC $= 0.5$) by a margin this sample size can treat as reliable. The one component whose removal produced a clear, directionally consistent effect was the Siamese tying constraint, and that effect was harmful: untying weights made the circuit rank pairs *worse* than chance. By that evidence, combining every component (tied weights, the swap test, and offset conditioning) is, at minimum, no worse than any simpler configuration tested, and is the only configuration this study did not find a reason to prefer against; it does not, however, constitute evidence that combining them yields a positive effect beyond what a zero-parameter classical baseline already achieves on the same data.

V. Conclusion

Copy-move forgery, the duplication and repositioning of a region within the same photograph, is difficult to expose precisely because the pasted region shares its sensor noise, lighting, and compression history with the rest of the image, leaving none of the cross-image inconsistencies a splicing detector could exploit. Reliable detection of this kind of tampering matters for digital forensics, journalism, legal evidence, and the everyday verification of images circulating on social media, and the central difficulty any such detector faces is telling a genuine duplication apart from the ordinary self-similarity that skies, foliage, and brickwork produce on their own. This work set out to test whether that problem could be addressed by a Siamese Swap-Test Quantum Neural Network, a pairwise patch comparator in which every trainable parameter is a quantum circuit rotation angle and no classical descriptor, matching algorithm, or voting stage sits anywhere in the decision path.

TABLE IX
Ablation results, n=200 synthetic pairs (100 positive, 100 negative).

Configuration	Params	AUC
Full (10-epoch checkpoint, as deployed)	58	0.5197
Without spatial offset filtering	58	0.5198
Without swap test (classical cosine)	0	0.5093
Full, tied (fresh, matched-scale retrain)	58	0.5133
Without Siamese (untied, matched-scale)	112	0.4688

The system operates by tiling an image into small patches, amplitude encoding each one into a quantum register, and scoring every admissible pair with a single shared circuit (Sections II–II-I). Three components carry that comparison. The Siamese design ties one weight tensor across both patch registers, which guarantees by construction that the circuit judges patch A against patch B the same way it would judge B against A, and does so with roughly half the parameters an untied pair of branches would need (Section II-F). The swap test is the mechanism that actually produces a similarity score, estimating the overlap between the two patches' quantum states through a single ancilla measurement rather than any classical distance function (Section II-G). The spatial offset register was added to address self-similarity directly in the circuit: it encodes the displacement between the two patches and couples that displacement into the same ancilla readout, so that, in principle, the measured similarity reflects geometric plausibility as well as pixel content, rather than delegating that judgement to an external classical vote the way traditional block-matching pipelines do (Section II-H).

The experimental findings, however, do not support that architecture as a working detector. Against the only two baselines actually evaluated under matched conditions, a parameter-free classical cosine-similarity computation and a trivial "mark everything forged" rule, the trained circuit shows no meaningful advantage: AUC of 0.883 against 0.887 for cosine similarity on CoMoFoD-derived pairs, and a best-achievable F1 of 0.0813 against 0.0771 for marking every pixel forged on real CoMoFoD images, using an oracle threshold no real deployment could actually choose in advance

(Section IV-A). A full end-to-end run on a synthetic scene with a known forgery made the underlying problem concrete rather than statistical: an entirely authentic, undoctored version of the same scene produced 1,492 circuit-flagged matches against 1,509 for the genuinely forged version, and both were assigned the identical maximum-confidence forged verdict (Section IV-B). Under every corruption tested, JPEG re-compression, Gaussian noise, rotation, scaling, blur, and brightness shift, the trained circuit's similarity score collapsed toward zero markedly faster than the classical cosine baseline did (Section IV-C). The ablation study found no configuration that cleared chance-level ranking by a reliable margin, and the one component whose removal produced a clear, consistent effect was the Siamese tying constraint, whose removal made matters worse rather than better (Section IV-D). No head-to-head comparison against the literature methods discussed in Section IV-B, block-based, keypoint-based, or deep-learning CMFD, was carried out, so no ranking against those methods specifically can be claimed in either direction; against the baselines this work did evaluate directly, the proposed method does not outperform existing approaches.

This does not make the exercise uninformative. The contribution here is a fully specified, fully quantum architecture for pairwise patch comparison, together with a proof, both analytical and measured, that a tied unitary applied after amplitude embedding is provably degenerate under a swap test unless the patch data is re-uploaded into the rotation angles at every layer, a design constraint any future work on quantum similarity circuits of this kind will need to respect (Section II-F). It also contributes an in-circuit mechanism for folding spatial-offset evidence into a swap-test readout, evaluated honestly rather than assumed to work, and an evaluation protocol, AUC against a matched classical baseline, full end-to-end detection on images with known ground truth, a six-condition robustness sweep, and a four-way component ablation, that is offered as a template for testing whether other quantum architectures proposed for this task actually earn their added complexity, rather than being reported only in isolation.

The limitations follow directly from what was measured. The offset register did not produce a detectable improvement in the ablation study (Section IV-D), which is consistent with a structural argument raised in Section II-H: genuine copy-move consistency is a property of many pairs agreeing on one displacement, and a circuit that scores one pair at a time has no way to observe that agreement across pairs it is never shown together. The circuit's measured fragility under ordinary image corruptions (Section IV-C) works directly against real-world deployment, where JPEG compression and mild geometric adjustment are the norm rather than the exception. Every result in this paper was produced on a classical statevector simulator rather than physical quantum hardware, and the shot-budget analysis in Section II-G indicates that resolving a useful threshold decision on real hardware, at the pair counts this architecture requires, would carry a cost well beyond what the gate count and circuit depth alone suggest. Finally, no copy of CoMoFoD was available on the machine this paper was written on, so this paper's own experiments in Section IV-A onward were run against procedurally generated synthetic images rather than real photographs, a substitution flagged explicitly wherever it applies rather than left for the reader to notice.

Several directions follow naturally from these limitations. The most immediate is repeating this paper's synthetic-image experiments, robustness sweep and ablation study included, directly against real CoMoFoD and COVERAGE images once available, to confirm whether the patterns observed here also hold on genuine photographs. A more fundamental direction is pursuing the honest replacement for the offset register's structural weakness identified above: quantum counting or amplitude estimation over a pair-index register, which could in principle observe agreement across many pairs at once rather than one at a time, though it requires QRAM not available to statevector simulation at this scale. Running the trained circuit on physical quantum hardware, rather than only estimating its shot budget analytically, would test whether the fragility measured in Section IV-C is a property of the architecture itself or an artefact of how it was simulated here. Finally, the same evaluation protocol applied in this paper to a cosine-similarity baseline could be run directly against the block-based, keypoint-based, and deep-learning methods of Section IV-B under one identical dataset split and metric definition, closing the one comparison this work explicitly left open.

REFERENCES

[1] J. Fridrich, D. Soukal, and J. Lukáš, "Detection of copy-move forgery in digital images," in *Proc. Digital Forensic Research Workshop (DFRWS)*, Cleveland, OH, USA, 2003.

[2] I. Amerini, L. Ballan, R. Caldelli, A. Del Bimbo, and G. Serra, "A SIFT-based forensic method for copy–move attack detection and transformation recovery," *IEEE Trans. Inf. Forensics Security*, vol. 6, no. 3, pp. 1099–1110, 2011.

[3] D. Tralić, I. Zupančić, S. Grgić, and M. Grgić, "CoMoFoD – New database for copy-move forgery detection," in *Proc. 55th Int. Symp. ELMAR*, Zadar, Croatia, 2013, pp. 49–54.

[4] B. Wen, Y. Zhu, R. Subramanian, T.-T. Ng, X. Shen, and S. Winkler, "COVERAGE – A novel database for copy-move forgery detection," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2016, pp. 161–165.

[5] Y. Wu, W. Abd-Almageed, and P. Natarajan, "BusterNet: Detecting copy-move image forgery with source/target localization," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 168–184.

[6] J. Bromley, I. Guyon, Y. LeCun, E. Säckinger, and R. Shah, "Signature verification using a 'Siamese' time delay neural network," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 1993, pp. 737–744.

[7] J. Biamonte, P. Wittek, N. Pancotti, P. Rebentrost, N. Wiebe, and S. Lloyd, "Quantum machine learning," *Nature*, vol. 549, pp. 195–202, 2017.

[8] H. Buhrman, R. Cleve, J. Watrous, and R. de Wolf, "Quantum fingerprinting," *Phys. Rev. Lett.*, vol. 87, no. 16, p. 167902, 2001.

[9] A. Pérez-Salinas, A. Cervera-Lierta, E. Gil-Fuster, and J. I. Latorre, "Data re-uploading for a universal quantum classifier," *Quantum*, vol. 4, p. 226, 2020.

[10] M. Schuld and N. Killoran, "Quantum machine learning in feature Hilbert spaces," *Phys. Rev. Lett.*, vol. 122, no. 4, p. 040504, 2019.